

A pseudocode of the conceptual algorithm is presented below:

```
# Get candidates for VendorID/ProductID from DCL DeviceSoftwareVersionModel
# schema (e.g. from a replica)
candidates = obtain_candidates_from_dcl(requestor.vendorID, requestor.productID)

def find_newer_version(candidates, requestor):
    # Ensure all candidate records are in monotonic increasing numerical order
    sort(candidates, key="softwareVersion", order="ASCENDING")

    currentCandidate = None

    for candidate in candidates:
        # Current version already newer: skip
        if requestor.softwareVersion >= candidate.softwareVersion:
            continue

        # Candidate requires higher version than already installed: skip
        if requestor.softwareVersion < candidate.minApplicableSoftwareVersion:
            continue

        # Candidate requires lower version than already installed: skip
        if requestor.softwareVersion > candidate.maxApplicableSoftwareVersion:
            continue

        # Update potential candidate since it is applicable
        currentCandidate = candidate

    if currentCandidate is not None:
        # Last value of currentCandidate is highest matching value
        candidateWasFound = True
        softwareVersionFound = currentCandidate.softwareVersion
    else:
        candidateWasFound = False
        softwareVersionFound = 0

    return (candidateWasFound, softwareVersionFound)
```

If **candidateWasFound** was true, then a version matching (**softwareVersionFound**) was found and its location and associated metadata can be found in the **DeviceSoftwareVersionModel** schema of the DCL.

While the **QueryImage** command MAY also contain the Location, HardwareVersion and Meta-